

文件系统 挑战性任务 实验报告

Thinking + challenge

Thinking 1.1

Q:

请思考并回答，在真实的操作系统中，如果使用内存中的地址作为文件控制块的字段，会出现哪些问题？为什么我们目前的操作系统不会出现这些问题？

A:

使用内存中的地址作为文件控制块的字段会带来诸多问题，一是该地址就只能被固定下来了，内存使用较为死板；二是该地址可能并未被分配或分配给其他用途了，直接使用可能会带来问题；三是万一目录的地址发生了改变，文件也不知道其发生了变化，误以为原地址中还是目录的地址。因为我们目前的操作系统内存利用率不高，且磁盘到内存地址是固定映射的，一一对应，并且我们的操作系统似乎好像没有回收过缓存，磁盘块一直待在内存中。

challenge 1.1

Q:

请在 user/include/fs.h 文件中找到 struct File 结构体的定义，并将其中的 f_dir 字段修改为 f_dir_block 和 f_dir_offset 两个字段，分别记录 当前文件所在的目录 的文件控制块 所在的磁盘块编号，以及当前文件所在的目录 的文件控制块 在磁盘块中的偏移。注意维护 f_pad 字段的长度，保证 struct File 结构体大小依然为 256 字节。

****A:****根据题目要求，我在代码中FCB中新增了f_dir_block和f_dir_offset分别记录当前文件所在的目录的文件控制块所在的磁盘块编号，以及当前文件所在的目录的文件控制块在磁盘块中的偏移,同时维护f_pad的长度。具体如下：

```
struct File
{
    char f_name[MAXNAMELEN]; // filename
    uint32_t f_size;          // file size in bytes
    uint32_t f_type;          // file type
    uint32_t f_direct[NDIRECT];
    uint32_t f_indirect;
    uint32_t f_dir_block;      // challenge 修改
    uint32_t f_dir_offset;     // challenge 修改
    char f_pad[FILE_STRUCT_SIZE - MAXNAMELEN - (5 + NDIRECT) * 4]; // challenge 修改
} __attribute__((aligned(4), packed));
```

Challenge 1.2

Q:

在修改完 struct File 结构体后，你需要同步修改所有使用到原 f_dir 字段的代码，保证整个操作系统能够正确运行。提示：你可以使用 grep 命令查找所有使用到原 f_dir 字段的位置。

A:

这个我主要先采用grep找到所有使用f_dir的位置，然后根据FCB中存储的f_dir_block和f_dir_offset,先用dir_addr()找到内存中的地址，再加上offset得到目录的地址，用struct File* 指向这块地址，得到的就是目录的指针了，直接替代就行了,类似于这样，过多，不一一列举了，其中用f_dir_block代表是否已分配，没有则f_dir_block为0，

```
if (f->f_dir_block)
{
    void *va = disk_addr(f->f_dir_block);
    va = (char *)va + f->f_dir_offset;
    file_flush((struct File *)va);
}
```

用它来替代原来的判断f_dir的逻辑。

Thinking 2.1

Q:

尝试删除 f->f_type == FTYPE_REG 的条件判断，即对于任意文件都执行上述第二步的操作。在删除前后分别使用 make test lab=6_2 && make run 执行启动 Shell ，依次执行 ls.b | cat.b > aaa.txt 和 cat.b aaa.txt 两条命令，观察输出结果有何变化，将输出结果记录在报告中。请自行调试程序，分析删除后程序运行出现问题的原因。

A:

```
//原始
$ ls.b | cat.b > aaa.txt
[00002803] pipecreate
[00003805] destroying 00003805
[00003805] free env 00003805
i am killed ...
[00004006] destroying 00004006
[00004006] free env 00004006
i am killed ...
[00003004] destroying 00003004
[00003004] free env 00003004
i am killed ...
[00002803] destroying 00002803
[00002803] free env 00002803
i am killed ...

$ cat.b aaa.txt
aaa.txt testarg.b cat.b pingpong.b testbss.b newmotd testpiperace.b testpipe.b
motd init.b num.b lorem testfdsharing.b testshell.sh script ls.b echo.b sh.b
halt.b testptelibrary.b
[00005004] destroying 00005004
[00005004] free env 00005004
i am killed ...
[00004803] destroying 00004803
[00004803] free env 00004803
i am killed ...
```

```
//删除
$ ls.b | cat.b > aaa.txt
[00002803] pipecreate
[00003805] destroying 00003805
[00003805] free env 00003805
i am killed ...
cannot mark pages as dirty
panic at cat.c:11: write error copying <stdin>: -10
[00004006] destroying 00004006
[00004006] free env 00004006
i am killed ...
[00003004] destroying 00003004
[00003004] free env 00003004
i am killed ...
[00002803] destroying 00002803
[00002803] free env 00002803
i am killed ...

$ cat.b aaa.txt
spawn cat.b: -10
[00004803] destroying 00004803
[00004803] free env 00004803
i am killed ...
```

因为我们删除了判断条件，导致现在目录也会被回收，但是目录中存储的是目录下文件的FCB，回收了之后，我们的文件指针就悬垂了，但是我们的文件并没有回收，导致我们再次访问该文件的FCB时就会触发地址错误导致崩溃。

Challenge 2.1

Q:

在观察到上述问题并找到原因后，请你修改相关代码，使得目录文件在关闭时，其文件内容中 所有磁盘块的缓存地址 到物理页的映射 能够被正常解除，且映射解除后不会对后续执行造成影响。在你的提交中，你需要上传已删除 `f->f_type == FTYPE_REG` 条件判断的版本。

****A:****我是仿照任务一通过在open结构体中添加打开的文件FCB所在的磁盘号和偏移量，在每次文件打开时填入，然后当访问FCB时发现没有分配，就通过read_block重新将文件的FCB放入缓存中，并填充到open字段里。

```

17  */
18  struct Open
19  {
20      struct File *o_file;
21      u_int o_fileid;
22      int o_mode;
23      struct Filefd *o_ff;
24      u_int o_file_block;
25      u_int o_file_offset;
26  };
27

```

```

filebno = rq->req_offset / BLOCK_SIZE;
if (!va_is_mapped(pOpen->o_file))
{
    void *va;
    read_block(pOpen->o_file_block, &va, 0);
    pOpen->o_file = (struct File *)((char *)va + pOpen->o_file_offset);
}

```

```

// Close a file.
void file_close(struct File *f)
{
    // Flush the file itself. Then unmap all blocks of the file.
    file_flush(f);
    u_int nblock = ROUND(f->f_size, BLOCK_SIZE) / BLOCK_SIZE;
    for (int i = 0; i < nblock; i++)
    {
        u_int diskbno;
        if (file_map_block(f, i, &diskbno, 0) < 0)
        {
            debugf("file_close: file_map_block failed\n");
            break;
        }
        unmap_block(diskbno);
    }
}

```

Thinking 3.1

Q:

尝试删除 `dir_lookup()` 函数中对 `f_dir_block` 和 `f_dir_offset` 字段的赋值。在删除前后分别使用 `make test lab=6_2 && make run` 启动 Shell，依次执行 `echo.b abc > aaa.txt`、`ls.b` 和 `cat.b aaa.txt` 三条命令，观察输出

结果有何变化，将输出结果记录在报告中。请自行调试程序，分析删除后程序运行出现问题的原因。提示：如果把所有命令中的 aaa.txt 换成 bbb.txt，输出结果会发生什么变化？这两个文件有什么区别？

A:

```
//原始
$ echo.b abc > aaa.txt
[00003004] destroying 00003004
[00003004] free env 00003004
i am killed ...
[00002803] destroying 00002803
[00002803] free env 00002803
i am killed ...

$ ls.b
aaa.txt testarg.b cat.b pingpong.b testbss.b newmotd testpiperace.b testpipe.b
motd init.b num.b lorem testfdsharing.b testshell.sh script ls.b echo.b sh.b
halt.b testptelibrary.b
[00004004] destroying 00004004
[00004004] free env 00004004
i am killed ...
[00003803] destroying 00003803
[00003803] free env 00003803
i am killed ...

$ cat.b aaa.txt
abc
[00005004] destroying 00005004
[00005004] free env 00005004
i am killed ...
[00004803] destroying 00004803
[00004803] free env 00004803
i am killed ...

$ echo.b abc > bbb.txt
[00006004] destroying 00006004
[00006004] free env 00006004
i am killed ...
[00005803] destroying 00005803
[00005803] free env 00005803
i am killed ...

$ ls.b
aaa.txt testarg.b cat.b pingpong.b testbss.b newmotd testpiperace.b testpipe.b
motd init.b num.b lorem testfdsharing.b testshell.sh script ls.b echo.b sh.b
halt.b testptelibrary.b bbb.txt
[00007004] destroying 00007004
[00007004] free env 00007004
i am killed ...
[00006803] destroying 00006803
[00006803] free env 00006803
i am killed ...
```

```
$ cat.b bbb.txt
abc
[00008004] destroying 00008004
[00008004] free env 00008004
i am killed ...
[00007803] destroying 00007803
[00007803] free env 00007803
i am killed ...

//删除
$ echo.b abc > aaa.txt
[00003004] destroying 00003004
[00003004] free env 00003004
i am killed ...
[00002803] destroying 00002803
[00002803] free env 00002803
i am killed ...

$ ls.b
aaa.txt testarg.b cat.b pingpong.b testbss.b newmotd testpiperace.b testpipe.b
motd init.b num.b lorem testfdsharing.b testshell.sh script ls.b echo.b sh.b
halt.b testptelibrary.b
[00004004] destroying 00004004
[00004004] free env 00004004
i am killed ...
[00003803] destroying 00003803
[00003803] free env 00003803
i am killed ...

$ cat.b aaa.txt
[00005004] destroying 00005004
[00005004] free env 00005004
i am killed ...
[00004803] destroying 00004803
[00004803] free env 00004803
i am killed ...

$ echo.b abc > bbb.txt
[00006004] destroying 00006004
[00006004] free env 00006004
i am killed ...
[00005803] destroying 00005803
[00005803] free env 00005803
i am killed ...

$ ls.b
aaa.txt testarg.b cat.b pingpong.b testbss.b newmotd testpiperace.b testpipe.b
motd init.b num.b lorem testfdsharing.b testshell.sh script ls.b echo.b sh.b
halt.b testptelibrary.b bbb.txt
[00007004] destroying 00007004
[00007004] free env 00007004
i am killed ...
[00006803] destroying 00006803
[00006803] free env 00006803
```

```
i am killed ...

$ cat.b bbb.txt
abc
[00008004] destroying 00008004
[00008004] free env 00008004
i am killed ...
[00007803] destroying 00007803
[00007803] free env 00007803
i am killed ...
```

可以看到删除前后aaa.txt文件版本时删除前有输出，而删除后没有输出，但是bbb.txt则是删除前后都有输出。这是因为aaa.txt本来就是在磁盘里的，他是在磁盘初始化的时候加入磁盘的，但是磁盘初始化并没有添加填充其FCB中目录的内容，导致其f_dir_block和f_dir_offset的初始值都是垃圾值，删除前在访问过程中对这两个字段进行重新填充，而删除后就仍是垃圾值，于是当然得不到正确的结果，但是bbb.txt原来是不在磁盘中的，他是通过file_create创建的，创建过程中自动填写了那两个字段，所以删除前后对于bbb.txt都无影响。

Challenge 3.1

Q:

在观察到上述问题并找到原因后，请你修改相关代码，使得 dir_lookup() 函数仅用来进行查找，不会在查找过程中对文件控制块的字段赋值。在你的提交中，你需要上传已删除 dir_lookup() 函数中对上述字段的赋值的版本。

A:

我主要是通过初始化磁盘写入文件和目录时顺便在其FCB中写入f_dir_block和f_dir_offset两个字段来确保不会初始化为垃圾值。

```
// Find the target among all 'File's in this block.
for (struct File *f = files; f < files + FILE2BLK; ++f)
{
    // Compare the file name against 'name' using 'strcmp'.
    // If we find the target file, set '*file' to it and set up i
    // field.
    /* Exercise 5.8: Your code here. (3/3) */
    if (strcmp(name, f->f_name) == 0)
    {
        *file = f;
        return 0;
    }
}
```

```
tools > C fsformat.c > ...
314 void write_file(struct File *dirf, const char *path)
333     else
337         strcpy(target->f_name, fname);
338
339     int offset, blockno;
340     target->f_size = lseek(fd, 0, SEEK_END);
341     target->f_type = FTYPE_REG;
342     get_file_pos((void *)dirf, &blockno, &offset);
343     target->f_dir_block = blockno;
344     target->f_dir_offset = offset;
345
```



```

ls > C:\fsformat.c > ...
3 void write_directory(struct File *dirf, char *path)
4 {
5     strcpy(pdir->f_name, basename(path), MAXNAMELEN - 1);
6     if (pdir->f_name[MAXNAMELEN - 1] != 0)
7     {
8         fprintf(stderr, "file name is too long: %s\n", path);
9         // File already created, no way back from here.
10        exit(1);
11    }
12    int offset, blockno;
13    pdir->f_type = FTYPE_DIR;
14    get_file_pos((void *)dirf, &blockno, &offset);
15    pdir->f_dir_block = blockno;
16    pdir->f_dir_offset = offset;
17    for (struct dirent *e; (e = readdir(dir)) != NULL;)
18    {
19        // ...
20    }
21}

22 void get_file_pos(void *ptr, int *blockno, int *offset)
23 {
24     for (int i = 0; i < NBLOCK; i++)
25     {
26         if (disk[i].type == BLOCK_FREE || disk[i].type == BLOCK_BOOT)
27         {
28             continue;
29         }
30         char *tmp = (char *)ptr;
31         char *f = (char *)(&disk[i].data);
32         if (tmp >= f && tmp < f + BLOCK_SIZE)
33         {
34             *blockno = i;
35             *offset = tmp - f;
36             return;
37         }
38     }
39     *blockno = 0;
40     *offset = 0;
41 }

```

Thinking 4.1

Q:

请自行设计一种检查地址是否已分配，并实现地址分配的方案。需要满足以下三个要求：一是可供分配的地址范围为 [0x10000000, 0x50000000)；二是保证已分配的地址能够正确被回收；三是保证较低的地址优先被分配。在设计好分配方案后，分析该方案占用的时间和空间，并据此简述为什么选取该方案。提示：你可以回顾

本学期实验课程中遇到的所有分配方案，例如页控制块和进程控制块的分配、`asid` 的分配、`fileid` 的分配、`fdnum` 的分配等。

****A:****我使用的是位图，其分配的时间复杂度是 $O(n)$ ，最坏的情况只剩下高地址没有被分配。回收和检查是否分配的时间复杂度是 $O(1)$ 。其空间开销是32KB，占用内存较小可以接受。基于较低的时间复杂度和良好的空间开销，我最终选择了位图，其实之前也考虑过用链表的，但是链表分配的时间复杂度也是 $O(n)$ ，且链表实现更为复杂，占用内存更大（为每一个磁盘块维护一个链表节点），于是就放弃了。

Thinking 4.2

Q:

请自行设计一种数据结构，用于存储磁盘块编号到缓存地址的映射，使得时间复杂度最小。注意存储并维护磁盘块编号不对应任何缓存地址的状态（即磁盘块未被缓存的状态）。

A:

在Thinking 4.1中已经回答过了。

Challenge 4.1

Q:

根据你上述的设计，实现磁盘块缓存地址的动态分配功能。

A:

我是通过维护两个数组来实现位图的，一个是`cache_bitmap`，用来记录内存空间有无被映射（每一位中1代表已经被映射，0则没有），另一个是`block_cache_va`，用来记录各个磁盘块映射到的内存地址（NULL或0代表没有或不合法，正常存储内存地址）。

```

3
4 struct Super *super;
5 uint32_t *bitmap;
6
7 #define CACHE_NPAGE ((CACHE_END - CACHE_START) / BLOCK_SIZE)
8 static uint32_t cache_bitmap[CACHE_NPAGE / 32];
9 #define MAX_BLOCKS 1024
10
11 static void *block_cache_va[MAX_BLOCKS];
12
13 void file_flush(struct File *);
14 int block_is_free(u_int);
15 int file_map_block(struct File *f, u_int filebno, u_int *diskbno, u_int alloc);
16
17 static inline uint32_t *get_bitmap(void)
18 {
19     return (uint32_t *)block_cache_va[2];
20 }
21
22 // Overview

```

```

24 // Hint: Use 'DISKMAP' and 'BLOCK_SIZE' to calculate the address.
25 int alloc_cache_page(u_int *va)
26 {
27     for (int i = 0; i < CACHE_NPAGE / 32; i++)
28     {
29         if (cache_bitmap[i] != 0xFFFFFFFF)
30         {
31             for (int j = 0; j < 32; j++)
32             {
33                 if (!(cache_bitmap[i] & (1 << j)))
34                 {
35                     cache_bitmap[i] |= (1 << j);
36                     *va = CACHE_START + (i * 32 + j) * BLOCK_SIZE;
37                     return 0;
38                 }
39             }
40         }
41     }
42     *va = 0;
43     return -E_CACHE_ERROR;
44 }

```

```
45
46 void free_cache_page(u_int va)
47 {
48     if (va >= CACHE_END || va < CACHE_START)
49     {
50         user_panic("the va is error for block cache!");
51         return;
52     }
53     int idx = (va - CACHE_START) / BLOCK_SIZE;
54     cache_bitmap[idx / 32] &= ~(1 << (idx % 32));
55 }
56
57 void *disk_addr(u_int blockno)
58 {
59     /* Exercise 5.6: Your code here. */
60
61     return block_cache_va[blockno];
62 }
63
```

```
63
64 u_int va_to_blockno(void *va, int *offset)
65 { /*1
66     if (offset)
67     {
68         *offset = (int)va & 0xffff;
69     }
70     void *page = (int)va & 0xFFFFF000;
71     for (int i = 0; i < super->s_nblocks; i++)
72     {
73         if (block_cache_va[i] == page)
74         {
75             return i;
76         }
77     }
78     return 0;
79 }
80
```

难点分析

本次的难点我个人觉得主要还是存在于第四个任务，其不仅要设计一个时间复杂度低的分配方案，还要对原来的分配方案进行一次彻彻底底的修改，修改的内容多，涉及到的东西广，一不小心就会漏掉一些。其中还有一些历史遗留的bug，像是file_block中的nblock没有向上取整，最终导致文件如果小于4KB就不会被置为脏页，这是历史遗留下来的bug,但是最终评测也会用到，于是我反复复盘这四个任务都没发现这个bug，最后还是claude找到的，就挺无语。

实验体会

这是最后一次写OS实验报告了，每一次实验都是近距离了解OS的过程，看着MOS从最开始的啥也不能到现在能用一些基本的指令和实现一些基本的功能，还是比较感慨万千的。最后也没什么想说的，希望OS的课程能够越办越好吧。